LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKURSIF

MUHAMAD MIRZA AGUSTA

109082500044

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Praktikum Modul 3 dan 4 ini berfokus pada konsep fungsi dan prosedur dalam pemrograman menggunakan bahasa Go. Membahas Teori function, prosedur, Pass by Value dan Pass by Reference

1. **Fungsi(Function)**
   Fungsi adalah blok kode yang menerima input (parameter) dan dapat mengembalikan nilai (return value). Dalam bahasa Go, fungsi didefinisikan dengan kata kunci func diikuti nama fungsi, parameter (jika ada), tipe data return (jika ada), dan blok kode. Fungsi memungkinkan pemisahan logika program menjadi bagian-bagian yang lebih kecil dan spesifik. Dengan menggunakan fungsi, pengembang dapat menghindari penulisan kode berulang dan meningkatkan efisiensi pengembangan perangkat lunak.
2. **Prosedur(Procedure)**
   Prosedur mirip dengan fungsi namun tidak mengembalikan nilai. Prosedur biasanya digunakan untuk mengeksekusi serangkaian instruksi tanpa perlu mengembalikan hasil ke pemanggil. Dalam Go, prosedur didefinisikan sebagai fungsi tanpa tipe return. Prosedur sangat berguna untuk tugas-tugas seperti mencetak output atau menampilkan menu, di mana hasil eksekusi cukup terlihat langsung tanpa perlu nilai kembalian.
3. **Pass by value dan pass by referance**

   parameter dalam fungsi dapat dikirim dengan dua cara:

- **Pass by Value**: Mengirim salinan nilai ke fungsi. Perubahan pada parameter di dalam fungsi tidak mempengaruhi variabel asli.
- **Pass by Reference**: Mengirim alamat memori (pointer) ke fungsi. Dengan menggunakan pointer, fungsi dapat mengubah nilai variabel asli. Kedua metode ini memberikan fleksibilitas kepada programmer untuk mengontrol apakah suatu fungsi boleh memodifikasi data asli atau hanya bekerja dengan salinannya.

4. **Fungsi rekursif dan iteratif**

   Fungsi dapat diimplementasikan secara iteratif (menggunakan perulangan) atau rekursiff (memanggil dirinya sendiri). Pemilihan metode tergantung pada efisiensi dan kompleksitas masalah. Pendekatan iteratif umumnya lebih hemat memori karena tidak memerlukan stack frame tambahan, sementara pendekatan rekursiff seringkali lebih intuitif masalah yang secara alami bersifat rekursiff seperti pencarian dalam struktur pohon atau perhitungan faktorial.

B. Guided

1. [Membuat baris bilangan dari n hingga 1 Base-case: bilangan == 1]

```go
package main

import "fmt"

func main() {
   var n int
   fmt.Scan(&n)
   baris(n)
}

func baris(bilangan int) {
   if bilangan == 1 {
      fmt.Println(1)
   } else {
      fmt.Print(bilangan, " ")
      baris(bilangan - 1)
   }
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\no 1\1.go"
10
10 9 8 7 6 5 4 3 2 1
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\no 1\1.go"
5
5 4 3 2 1
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
```

Penjelasan :

Program ini dimulai dari fungsi main yang bertugas menerima satu input bilangan bulat dari pengguna, kemudian memanggil fungsi baris dengan bilangan tersebut sebagai parameter.Fungsi baris dirancang secara rekursiff. Ia akan terus memanggil dirinya sendiri dengan nilai yang terus berkurang satu, hingga mencapai kondisi dasar (base case) yaitu ketika bilangan bernilai 1. Pada setiap pemanggilan sebelum kondisi dasar tercapai, program mencetak bilangan saat ini diikuti dengan spasi. Ketika akhirnya bilangan bernilai 1, program mencetak angka 1 dan menghentikan seluruh rangkaian pemanggilan rekursiff.Hasilnya, program mampu mencetak deret bilangan menurun dari n hingga 1 dalam satu baris, seperti "5 4 3 2 1" jika inputnya adalah 5

2. [Menghitung hasil penjumlahan 1 hingga n Base case : n ==1]

```
package main

import "fmt"

func main() {
   var n int
   fmt.Scan(&n)
   fmt.Println(penjumlahan(n))
}

func penjumlahan(n int) int {
   if n == 1 {
      return 1
   } else {
      return n + penjumlahan(n-1)
   }
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\NO 2\2.go"
10
55
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\NO 2\2.go"
5
15
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> S
```

Penjelasan :

Program ini adalah implementasi rekursiff untuk menghitung penjumlahan bilangan bulat dari 1 hingga n (atau sering disebut sebagai fungsi penjumlahan deret aritmatika). Fungsi main bertugas menerima input bilangan bulat n dari pengguna, kemudian memanggil fungsi penjumlahan(n) dan mencetak hasil yang dikembalikan. Fungsi penjumlahan dirancang dengan logika rekursiff yang sangat klasik: jika nilai n sama dengan 1, fungsi akan mengembalikan 1 sebagai kondisi dasar (base case). Namun jika n lebih besar dari 1, fungsi akan mengembalikan nilai n ditambah dengan hasil pemanggilan fungsi penjumlahan(n-1).Proses rekursiff ini akan terus berlanjut hingga mencapai n = 1. Sebagai contoh, jika pengguna memasukkan angka 5 Hasil akhirnya adalah 5+4+3+2+1 = 15.

.

3. [Mencari dua pangkat n atau 2^n -case: n == 0]

```
package main

import "fmt"

func main() {
   var n int
   fmt.Scan(&n)
   fmt.Println(pangkat(n))
}

func pangkat(n int) int {
   if n == 0 {
      return 1
   } else {
      return 2 * pangkat(n-1)
   }
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\NO3\3.go"
15
32768
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\guided\NO3\3.go"
5
32
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
```

Penjelasan:

Program ini menerima input bilangan bulat n dari pengguna, kemudian mencetak hasil perhitungan 2^n.Fungsi pangkat dirancang dengan logika rekursiff yang sederhana. Kondisi dasarnya adalah ketika n == 0, fungsi mengembalikan 1 karena 2^0 = 1. Untuk n lebih besar dari 0, fungsi mengembalikan 2 dikalikan dengan hasil pemanggilan pangkat(n-1). Proses ini terus berulang hingga mencapai kondisi dasar. Sebagai contoh, jika n = 4, program akan menghitung 2 × 2 × 2 × 2 × 1 = 16.Program ini merupakan contoh edukatif yang baik untuk memahami konsep rekursif, bagaimana fungsi memanggil dirinya sendiri, serta cara memecah masalah menjadi sub-masalah yang lebih kecil.

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilaisuku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapatdiformulasikan $S_n = S_{n-1} + S_{n-2}$ .Berikut ini adalah contoh nilai deret fibonacci hingga sukuke-10

Jawaban :

```
package main

import "fmt"

func fibonacci(n int) int {

    if n == 0 {
        return 0
    }
    if n == 1 {
        return 1
    }

    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    var n int

    fmt.Print("Masukkan suku ke-n yang ingin dicari: ")
    fmt.Scan(&n)

    if n < 0 {
        fmt.Println("Masukkan bilangan non-negatif!")
        return
    }

    fmt.Printf("\nNilai suku ke-%d dari deret Fibonacci
adalah: %d\n\n", n, fibonacci(n))

    fmt.Println("Deret Fibonacci hingga suku ke-n:")

    fmt.Print("n     |")
    for i := 0; i <= n; i++ {
        fmt.Printf(" %d |", i)
    }
    fmt.Println()

    fmt.Print("-------")
```

```
    for i := 0; i <= n; i++ {
        fmt.Print("----")
    }
    fmt.Println()

    fmt.Print("S_n    |")
    for i := 0; i <= n; i++ {
        fmt.Printf(" %d |", fibonacci(i))
    }
    fmt.Println()
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\UNGUIDED\no1\1.go"
Masukkan suku ke-n yang ingin dicari: 10

Nilai suku ke-10 dari deret Fibonacci adalah: 55

Deret Fibonacci hingga suku ke-n:
n     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
--------------------------------------------------
S_n   | 0 | 1 | 1 | 2 | 3 | 5 | 8 | 13 | 21 | 34 | 55 |
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
```

Penjelasan :

Program ini adalah implementasi deret Fibonacci menggunakan rekursif dalam bahasa Go. Program menerima input bilangan bulat non-negatif n dari pengguna, kemudian menampilkan nilai suku ke-n dari deret Fibonacci beserta seluruh deret hingga suku ke-n dalam bentuk tabel yang rapi.Fungsi fibonacci dirancang secara rekursiff dengan dua kondisi dasar: jika n = 0, fungsi mengembalikan 0; jika n = 1, fungsi mengembalikan 1. Untuk nilai n lebih besar dari 1, fungsi mengembalikan hasil penjumlahan dari fibonacci(n-1) dan fibonacci(n-2). Logika ini mengikuti definisi matematis deret Fibonacci di mana setiap suku merupakan jumlah dari dua suku sebelumnya.Program ini juga dilengkapi dengan validasi input untuk menolak bilangan negatif. Setelah menghitung nilai suku ke-n, program menampilkan tabel yang menunjukkan indeks suku (n) dan nilai suku (S_n) untuk setiap bilangan dari 0 hingga n. Tabel ini dicetak dengan format yang rapi menggunakan garis pemisah, memudahkan pengguna untuk melihat seluruh deret secara lengkap.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursiff untuk menampilkan faktor bilangan dari

suatu N, atau bilangan yang apa saja yang habis membagi N..

Jawaban :

```
package main

import "fmt"

func tampilkanFaktor(n, current int) {

   if current > n {
      return
   }

   if n%current == 0 {
      fmt.Printf("%d ", current)
   }

   tampilkanFaktor(n, current+1)
}

func main() {
   var n int

   fmt.Print("Masukkan bilangan bulat positif N: ")
   fmt.Scan(&n)

   if n <= 0 {
      fmt.Println("Masukkan bilangan bulat positif!")
      return
   }

   fmt.Printf("Faktor dari %d adalah: ", n)
   tampilkanFaktor(n, 1)
   fmt.Println()
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\UNGUIDED\no2\2.go"
Masukkan bilangan bulat positif N: 5
Faktor dari 5 adalah: 1 5
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\UNGUIDED\no2\2.go"
Masukkan bilangan bulat positif N: 12
Faktor dari 12 adalah: 1 2 3 4 6 12
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
```

Penjelasan :

Program ini menerima input bilangan n dari pengguna, kemudian menampilkan seluruh bilangan yang dapat membagi n tanpa sisa.Fungsi tampilkan Faktor dirancang dengan dua parameter: n (bilangan yang dicari faktornya) dan current (bilangan pembagi yang sedang diperiksa, dimulai dari 1). Logika rekursiffnya sederhana: jika current melebihi n, fungsi akan berhenti (base case). Jika current habis membagi n (ditandai dengan n % current == 0), maka current dicetak sebagai faktor. Setelah itu, fungsi memanggil dirinya sendiri dengan parameter current+1 untuk melanjutkan pemeriksaan ke bilangan berikutnya.Program ini dilengkapi dengan validasi input untuk memastikan pengguna hanya memasukkan bilangan bulat positif. Sebagai contoh, jika pengguna memasukkan angka 12, program akan mencetak "Faktor dari 12 adalah: 1 2 3 4 6 12"

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursiff untuk menampilkan barisan bilangan ganjil.

Jawaban :

```
package main

import "fmt"

func tampilkanGanjil(current, n int) {

   if current > n {
      return
   }

   if current%2 != 0 {
      fmt.Printf("%d ", current)
   }

   tampilkanGanjil(current+1, n)
}

func main() {
   var n int

   fmt.Print("Masukkan bilangan bulat positif N: ")
   fmt.Scan(&n)

   if n <= 0 {
      fmt.Println("Masukkan bilangan bulat positif!")
      return
   }

   fmt.Printf("Bilangan ganjil dari 1 hingga %d adalah: ", n)
   tampilkanGanjil(1, n)
   fmt.Println()
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\UNGUIDED\no3\3.go"
Masukkan bilangan bulat positif N: 5
Bilangan ganjil dari 1 hingga 5 adalah: 1 3 5
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I
\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 5\UNGUIDED\no3\3.go"
Masukkan bilangan bulat positif N: 20
Bilangan ganjil dari 1 hingga 20 adalah: 1 3 5 7 9 11 13 15 17 19
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA>
```

Penjelasan :

Program ini menerima sebuah bilangan bulat positif sebagai input, kemudian menampilkan seluruh deret yang dihasilkan hingga mencapai angka 1 sesuai dengan

aturan yang telah ditentukan.Aturan jika bilangan saat ini genap, maka bilangan berikutnya adalah hasil bagi dengan 2 (n/2); jika bilangan ganjil, maka bilangan berikutnya adalah 3 dikali bilangan ditambah 1 (3n+1). Proses ini terus berulang hingga mencapai angka 1. menyatakan bahwa berapapun bilangan awalnya, proses ini akan selalu berakhir di angka 1. ini menggunakan fungsi cetakDeret yang menerima parameter bilangan dan melakukan perulangan dengan kondisi n != 1. Di setiap iterasi, program mencetak nilai saat ini, kem udian memperbarui nilai sesuai aturan di atas. Setelah loop selesai (n = 1), program mencetak angka 1 sebagai akhir deret.

D. Kesimpulan

Berdasarkan praktikum Modul 3 dan 4 ini, dapat disimpulkan bahwa pemahaman tentang fungsi dan prosedur merupakan fondasi penting dalam pengembangan perangkat lunak yang terstruktur. Melalui serangkaian program yang telah dibuat, praktikum ini berhasil mendemonstrasikan bagaimana fungsi dapat digunakan untuk memisahkan logika program menjadi bagian-bagian yang lebih kecil dan spesifik, sehingga kode menjadi lebih mudah dipahami, diuji, dan dipelihara. Konsep pass by value dan pass by reference yang diperkenalkan memberikan pemahaman mendalam tentang bagaimana data dikirim dan dimanipulasi dalam fungsi, di mana pass by value melindungi data asli dari perubahan yang tidak diinginkan sementara pass by reference memungkinkan fungsi untuk memodifikasi data asli secara langsung dan efisien. Program demonstrasi dengan fungsi tambahvalue dan tambahreference secara jelas memperlihatkan perbedaan kedua metode tersebut melalui output yang dihasilkan.

Program deret Collatz yang diimplementasikan dengan prosedur cetakDeret menunjukkan bagaimana prosedur dapat digunakan untuk mengeksekusi serangkaian instruksi tanpa perlu mengembalikan nilai, sementara program kalkulator bangun datar dengan sistem menu interaktif membuktikan bahwa fungsi dan prosedur dapat dikombinasikan dengan struktur kontrol untuk menciptakan aplikasi yang user-friendly. Secara keseluruhan, praktikum ini tidak hanya mengajarkan aspek teknis pemrograman tetapi juga menekankan pentingnya perancangan kode yang baik, efisiensi algoritma, validasi input, dan penerapan konsep pemrograman dalam menyelesaikan permasalahan matematika dan dunia nyata, seperti yang terlihat pada program permutasi kombinasi dan komposisi fungsi.